

Executive Summary

The Brudo/Brullama transcript and code reveal a shift from abstract governance concepts to concrete **computer-run pipelines** that enforce “constitutional” state-transition rules. The invention centers on a **multi-stage transition pipeline** where admitted partial observations yield a *Consequence-Selective* state transition (admit or withhold) with typed receipts, rather than simply restoring a checkpoint. Key novel elements include: (1) a **Consequence-Selective State Transition Pipeline** whose first stage is structural state admission and later stages explicitly gate execution by consequence determinacy; (2) **Replay-Time Re-Admission**, where replay re-evaluates admission predicates instead of blindly trusting prior results; (3) **Consequence-Tag-Selective Validation**, where validators match declared consequence tags to actual consequences, emitting fine-grained receipts; (4) **Identity-Bound Semantic Result Admission**, binding a declared result schema/contract identity into the job identity and requiring semantic conformance before admitting a generative output; and (5) a **Minimal Lawful-Information Runtime** that preserves only those distinctions (observations or world hypotheses) strictly needed to ensure the authorized consequence, neither collapsing nor retaining more state than necessary. Each is treated as a candidate for patenting. Our analysis below details each candidate’s technical core, existing implementation evidence, potential prior art, and patent strategy.

Notably, each candidate is framed as a **technical improvement to computer functionality** (e.g. by ensuring correct state-machine behavior or data integrity) rather than a mere abstract governance rule [21†L3390-L3398] [21†L3454-L3463]. This aligns with USPTO guidance that invention claims should focus on concrete computing improvements [21†L3390-L3398]. We compare each candidate against prior-art categories (e.g. provenance models like W3C PROV [28†L50-L58], supply-chain attestations like in-toto [34†L194-L199], counterexample-guided abstraction (CEGAR) for partial-observation models [32†L82-L89], and sensor-selection literature [27†L103-L111]) to pinpoint novelty and risks. Finally, we propose an experimental plan to close enablement gaps (P3→P5 prompts, code/tests to add, 2×2 assays, metrics) and outline provisional claims for the top two candidates, with mermaid diagrams illustrating the pipeline flow, identity relationships, and the P3–P5 timeline.

Patent Candidates (one-sentence summaries):

- **Consequence-Selective State Transition Pipeline:** A concrete multi-stage state machine pipeline that admits or withholds execution based on partial observations and authorized “consequences,” producing typed receipts (distinguished from abstract procedural rules) – novel because it binds multiple validation stages (admission, replay, consequence filtering, disposition) into a single execution pipeline; risk is moderate due to overlap with state-machine and workflow patents.
- **Replay-Time Re-Admission:** A mechanism where replay of a saved computation *recomputes* admission predicates (instead of trusting stored admission), ensuring correctness; novel as a runtime behavior in execution pipelines, but risk is high since similar ideas appear in replication systems (e.g. transactional replay without ordering [24†L49-L57]).
- **Consequence-Tag-Selective Validation:** A validation engine that filters and intersects “distinctions” between worlds by their consequence tags and declared consequences, emitting per-distinction

receipts; novel in its focus on consequence-relative difference checking, risk moderate (some similarity to partial-observability verification).

- **Identity-Bound Semantic Result Admission:** Binding a declared result contract (e.g. JSON schema) into the work identity and requiring the *exact* declared contract (not just any valid output) to admit a generative model's result; novel because job identity includes contract identity and acceptance requires semantic checks, risk moderate (structured-output validation is known, but identity-binding is new).
- **Minimal Lawful-Information Runtime:** A runtime rule that preserves *only* those distinctions (inferred world-states or observations) strictly needed to maintain the authorized consequence, refusing to collapse or preserve beyond that minimal set; novel as a bidirectional “distinction-preservation” mechanism, risk high due to related work on belief states and partial observability.

Below we analyze each in depth.

Candidate	Novelty (1–5)	Prior-Art Risk (1–5)	Implementation Support (score)	Action
Consequence-Selective State Transition	5	3	4	File
Replay-Time Re-Admission	5	4	3	Hold/ Merge
Consequence-Tag-Selective Validation	4	3	3	File
Identity-Bound Semantic Result Admission	4	3	3	File
Minimal Lawful-Information Runtime (Selective Distinction Preservation)	4	5	2	Hold

1. Consequence-Selective State Transition Pipeline

- **Technical Problem:** Conventional state transitions (e.g. restoring a checkpoint or running to completion) trust previous admissions and lack a flexible pipeline to enforce authorized outcomes. The problem is to execute code based on partial evidence but only commit a transition if it leads to a *single authorized consequence*, and otherwise withhold.
- **Solution (Minimal Technical Combination):** A structured pipeline with stages: (a) **structural admission** (check admissibility of state/operation given partial evidence), (b) **witness validation** (verify evidence freshness/integrity), (c) **consequence-distinction filtering** (compute consequences of all compatible states and group by outcome), (d) **consequence admission gate** (allow execution only if exactly one admissible consequence block matches the requested consequence), and (e) **disposition/receipt** (emit PASS/FAIL or admit-withhold with typed receipts). Crucially, the pipeline *replays admission* and recomputes distinctions rather than trusting cached decisions.
- **Implementation Support:** The current code implements this pipeline: in `meaning/collision.go` (lines 235–262), `BuildConsequenceQuotient` groups compatible worlds by their consequence value (excluding invalid bases), and `GateConsequenceQuotient` enforces: **one** consequence block $\Rightarrow$ admit; **>1** $\Rightarrow$ withhold; **excluded/unresolved bases** $\Rightarrow$ unresolved **[24†L49-L57]**. Test `TestQuotientGateAdmitsOnlyOneUnexcludedBlock` demonstrates the behavior (switching a

block's consequence changes admit→withhold). Thus the pipeline is concrete: input states → quotient → gate decision. Negative control: feeding two conflicting consequence blocks yields withholding (tested).

- **Technical Effect:** Improves reliability of state management and execution by guaranteeing that only fully-admissible, single-consequence transitions proceed. This is a concrete computing improvement (similar to how specialized state machines improve system correctness [21†L3390-L3398]) because it adds enforcement logic and receipts that conventional systems lack. It also avoids silent failures/commits under uncertainty.
- **Nearest Prior Art:** The idea resembles **replicated state machines** and consistency systems that execute/validate/replay transactions. For example, US 12657207 (Pat. June 2026) describes a replicated SMR pipeline that “executes, validates (replay) then merges transactions” without pre-ordering [24†L49-L57] . Our pipeline similarly decouples execution from commit via replay checks. Other relevant areas include workflow engines with gating logic, and partial-observation model-checking (CEGAR-like refinement [32†L82-L89]). However, none tie the gating to “consequences” of states. **W3C PROV** models provenance of activities/entities but not execution gating [28†L50-L58] .
- **Distinguishing Claim Language:** e.g. “A computer-implemented method comprising: obtaining a snapshot of system state and partial observations; computing a set of compatible worlds; partitioning the worlds by their consequences; and enabling execution only if exactly one consequence block matches the requested consequence.” Emphasize *consequence-directed grouping* and gating. Unlike prior systems (e.g. transactional pipelines [24†L49-L57]), this explicitly filters by *authorized consequence*.
- **Enablement Gaps:** The code covers the gating logic but full disclosure of structural admission predicates and witness-record steps is incomplete. We should define what evidence is needed in each stage, and how receipts are constructed end-to-end (some receipts types exist, but full schema should be written). Also, handling of nondeterministic or partially failing evidence should be clarified.
- **Claim-Scope Risks:** Broad claims on “state transition pipelines” could be seen as abstract. Risk of overclaiming if not tied to concrete computation steps (MPEP §2106 emphasizes practical application [21†L3390-L3398]). Must clearly include computer-implementation details (data structures, receipts).
- **Status: FILE_CANDIDATE** – High novelty and implementation support. This is the central new mechanism.

2. Replay-Time Re-Admission

- **Technical Problem:** Traditional replay (e.g. in checkpoint/rollback or event-sourcing) often trusts that a past admission decision remains valid, which can hide errors. The problem is ensuring that replayed execution is re-checked against current admissibility policies to prevent silent acceptance of an invalid state or policy change.
- **Solution:** During replay of a saved state or candidate result, **recompute** all admission predicates (structural checks, world resolution) instead of using cached admission receipts. In other words, treat replay as a fresh decision point: if evidence admissibility has lapsed or new rules apply, replay can yield a different decision.
- **Implementation Support:** The conversation notes that “the implementation explicitly says replay recomputes admission rather than trusting stored results.” (This appears in the code’s comments and pipeline design, though full code isn’t shown in transcripts.) A test scenario: if a basis’s provenance is later found invalid, then replay of the same world set should now fail admission. Negative control: compare replay under old vs. new policy (not yet coded).

- **Technical Effect:** Increases system integrity by preventing outdated or invalid assumptions during replay. This is a concrete robustness improvement (analogous to “re-validation” in safe state restoration). It trades off performance for correctness.
- **Nearest Prior Art:** Many database and replication systems require determinism, but some (like the patent above [24†L49-L57]) allow concurrent execution/replay with validation. The closest is transactional SMR [24], which “replays” to verify, but our context is state admission policies, not just logs. The concept is related to *idempotency* and *resilience* in distributed systems. Formal methods like CEGAR re-abstract on replay [32†L82-L89] , and dynamic proof systems re-check constraints, but tying to user-defined admission is new.
- **Distinguishing Language:** E.g.: “on replay of a checkpoint or candidate, re-evaluate the admission conditions and only proceed if they still hold.” Contrast with “conventional restore uses cached commit flags.” Claim emphasis: “regenerating admission receipts from original inputs.”
- **Enablement Gaps:** Need to specify what it means to “replay” (time, determinism). Also need a mechanism to prevent infinite loops if admission fails (e.g. fallback or alarm).
- **Claim Risks:** Might be seen as an obvious integrity check or “error-checking.” Must tie to computer-execution flow. Broader claims might overlap with generic “reexecution under new policy.”
- **Status: HOLD/MERGE** – The idea is strong but overlaps prior art in replication; it may be merged into the pipeline patent or made dependent.

3. Consequence-Tag-Selective Validation

- **Technical Problem:** When multiple candidate “worlds” (state hypotheses) yield different outcomes (consequences), it is necessary to identify and segregate distinctions (evidence variables) that determine each outcome. The problem is efficiently validating only those distinctions relevant to the *declared* consequences and generating receipts per-distinction.
- **Solution:** Introduce a **consequence-relative equivalence** or quotient: group worlds by the declared consequence tags and check “derivability” per group. The validator intersects each world’s declared consequences with the target consequence, then issues per-distinction pass/fail receipts. In code: given `ConsequenceBasis` (the logic that derived a candidate), only worlds deriving the same consequence and admissible basis stay in one block. Receipt types (e.g. `ConsequenceDeterminacyReceipt`) bind a consequence type, evidence digest, basis, and status.
- **Implementation Support:** The code builds a `ConsequenceQuotient` (meaning/collision.go lines 159–187) grouping reality IDs by `DerivedByConsequence` . It then uses `GateConsequenceQuotient` to admit only one matching block. The test `TestSameObservationDispatchesDifferentConsequencesDifferently` verifies that the same observation class C1 vs C2 causes different gate outcomes (demonstrating block filtering). Negative control: if two worlds share a consequence tag but one has an invalid derivation basis, it is excluded (test `TestConsequenceQuotientExcludesUnadmissibleBases`).
- **Technical Effect:** Focuses validation effort only on relevant evidence, improving efficiency and correctness (it avoids “collapsing” across irrelevant distinctions). In effect, it implements a tailored equivalence relation for execution. This is akin to partial-observability filtering in verification, a technical advance in runtime state tracking.
- **Nearest Prior Art:** Related to **belief-state abstraction** and *partial equivalence classes*. Research on partial-observable model checking (e.g. POMDP abstraction [32†L82-L89]) and sensor selection prioritize distinctions. However, existing work does not couple equivalence classes explicitly to user-declared consequences. The novelty is linking evidence filtering to semantic tags rather than generic observation equivalence.

- **Distinguishing Language:** Claims should stress “intersection of consequence tags” or “filtering by declared consequence.” For example: “generating evidence-based distinction receipts only for those semantic distinctions that intersect the requested consequence tag, and providing failure receipts otherwise.” This is more specific than general “data filtering.”
- **Enablement Gaps:** We should formalize “consequence equivalence” (the quotient definition) and clearly tie it to the data model (how tags are declared in basis). Also define how receipts incorporate these tags.
- **Claim Risks:** Could be conflated with “schema-based validation” or “type checking.” Must emphasize the multi-world, evidence-driven context.
- **Status:** FILE_CANDIDATE – Technically concrete and supported by code/tests.

4. Identity-Bound Semantic Result Admission

- **Technical Problem:** Generative AI systems often declare a result schema (JSON contract) but do not enforce it at runtime, leading to semantic drift across nondeterministic outputs. The problem is to ensure that the declared result contract is treated as an *identity-bound admission predicate* – i.e., the exact declared schema (or contract) must match the output for the job to complete.
- **Solution:** Include the **ResultSchema identity** (e.g. a hash of the schema) in the *Job Identity*. Require that any output considered “complete” must parse and *fully satisfy* that exact schema. Admission ($\text{Admit}(r, j)$) is defined as $\text{parse}(r)$ AND $\text{conforms}(r, \text{Contract_j})$ AND $\text{semanticPredicates}(r, j)$. In practice, the system must tie the contract identity to the result’s admission logic. If the output is syntactically valid JSON but deviates from the declared schema, the job is not admitted (fail-closed). Receipt data should bind the job ID, contract ID, and results of each predicate.
- **Implementation Support:** The issue arose when low-capacity LLM outputs truncated JSON (fail) but higher-capacity produced valid JSON that **did not include all contract keys** (“six-key” vs “nine-key” contract). The current system carries `ResultSchema` in the work identity but does not enforce it at the admission gate (as evidenced by both valid-but-contracted-incomplete outputs being admitted). In code, `ResultSchema` appears in `RunReport` and `CandidateFinding` but is not used by `FindingAdmission`. We must add logic to check `ResultSchema` at admission time. Negative control: vary model capacity to produce structurally different valid outputs; the system currently treats both as “COMPLETED” without enforcing the promised schema.
- **Technical Effect:** Ensures data integrity of generative pipelines, preventing silent contract violations. This improves system security and trustworthiness (analogous to enforcing API type contracts). It’s a technical safeguard that has tangible effects on system correctness.
- **Nearest Prior Art:** Structured-output pipelines (e.g. prompt+validate with JSON schema) are known, and tools like W3C PROV or in-toto allow claiming provenance of data vs schema validity [28†L50-L58] [34†L194-L199]. However, tying the *identity* of the schema into the work request and requiring semantic conformance as an admission predicate appears novel. There is research distinguishing *schema validity* from *semantic correctness*, but rarely combining identity-binding and runtime admission. No known patent specifically enforces a declared schema via the request identity in a generative AI pipeline.
- **Distinguishing Language:** E.g.: “A method where a job request includes an identity-bound result contract; generating output and admitting completion **only if** the output parses to valid JSON *and* fully conforms to the declared contract identity.” Highlight that the *contract identity* is part of the work ID and enforces admission, unlike prior schema-checker components.

- **Enablement Gaps:** Need to specify how to handle partial success (e.g. missing fields) – presumably by rejection with a detailed receipt. Also clarify how to update/record the contract identity (e.g. include schema hash in receipts). Tests needed for “contract mismatch” cases.
- **Claim Risks:** Patent examiners might see this as obvious schema validation. We must emphasize the *identity-binding* (the schema’s hash is part of the job identity) and the admission-semantics. Otherwise it could be rejected as known data validation.
- **Status: FILE_CANDIDATE** – Novel and supported by observed bug; close to prior art but with a unique identity-binding twist.

5. Minimal Lawful-Information Runtime (Selective Preservation)

- **Technical Problem:** In partially-observable systems, it is tempting to keep either “maximally detailed” or “fully abstract” state representations. The challenge is to preserve exactly those distinctions in the state that are *necessary* to keep the authorized consequence invariant, and no more. In other words, collapse equivalent worlds up to that coarse but preserve whatever fine detail might affect admissibility of the consequence.
- **Solution:** Define a runtime rule: two states or observations can be merged (regarded as equivalent) only if doing so *provably* cannot change the admitted consequence. Conversely, any distinction that could affect a future authorized outcome must be preserved. Technically, this means computing a **consequence-basis admissibility equivalence** ($r_a \approx_{\{C,B,P\}} r_b$ iff they yield the same C-outcome *and* both are admissible under basis B,P). The system should actively collapse or separate worlds based on this relation and should “simulate” candidate merges under all relevant outcomes. This enforces neither maximal nor minimal information but a middle ground tailored to consequences.
- **Implementation Support:** The transcript outlines this principle (collapse only distinctions safe for the consequence). The code already differentiates *consequence determinacy* vs *derivation admissibility*. However, full realization requires automated searches for “Minimum Sufficient Observation” (the least observation that refines the consequence image), and a non-interference check: if an observation q never changes $\Gamma_C(E)$, it should not be acquired. These are not yet implemented. Negative control: forcing two distinct worlds into one would yield wrong consequence if one derivation was inadmissible (see proposed “killer assay” below).
- **Technical Effect:** This yields an optimal runtime information management: reducing memory and computation by discarding irrelevant distinctions while avoiding unsafe merges. It draws on formal verification (CEGAR) principles to achieve correctness with minimal state. It improves efficiency and correctness of the monitoring system.
- **Nearest Prior Art:** The concept echoes *belief-state minimal sufficient statistics* in POMDPs [32†L82-L89] and sensor-selection for observability [27†L103-L111]. Prior work on “observable quotient” or “state minimization for partial systems” exists, but typically without consequence-based gating. It also relates to *supervaluation semantics* in AI. We found no art combining all aspects: *consequence-relative equivalence + admissibility + minimal observability* in one runtime.
- **Distinguishing Language:** Claim something like “collapsing candidate states only when provably safe for the requested consequence and its admission; otherwise preserving distinctions.” Emphasize the two-way error rule (“refuse collapse-too-much or preserve-too-much”). This is finer than “information minimization” generally.
- **Enablement Gaps:** We must formalize how to compute the equivalence (possibly via testing all outcomes of an observation, i.e. noninterference theorem). Implementing a search for “minimum missing observation” as described is needed. Also address how this interacts with dynamic knowledge updates.

- **Claim Risks:** Very abstract-sounding; examiners might consider it another belief-state trick. Must tie to concrete runtime checks and code. Broad claims on “minimal information runtime” are risky.
- **Status: HOLD** – Interesting but highly overlapping with known partial-observability theory [27†L103-L111] ; better to treat as a research goal or merge with Consequence pipeline claims.

2×2 Assay (Negative Control)

As a conceptual “killer assay” for the above: consider two histories `W1` and `W2` with **identical observed evidence** and **same candidate result** but different underlying origin. **If one world’s derivation fails under current policies and the other succeeds, they must not be merged.** For example:

Observations (identical)	Histories	Consequence	Derivation Admission	Dispatch
same (<code>E</code>)	W1, W2	singleton	W1:PASS, W2:FAIL	WITHHOLD
(world identity unresolved)		(same c)		

This shows that *consequence determinacy* $\neq$ *derivation admissibility*. The implementation must catch such cases (withhold) even though the consequence is determinate.

Conversely, if two different requested consequences share the same observation set, the system must distinguish them. The existing test

`TestSameObservationDispatchesDifferentConsequencesDifferently` does this for C1 vs C2: same world set but different target consequence leads to ADMIT vs WITHHOLD, proving “same evidence does NOT imply same dispatch.”

Patent Strategy and Prior Art

We emphasize **technical improvements** in each claim (e.g. in how state is managed or how execution correctness is enforced) [21†L3390-L3398] [21†L3454-L3463] . Potential prior-art sources include:

- **W3C PROV Data Model [28†L50-L58]** : Models provenance of entities/activities; analogous to our “lineage” but our claims go beyond generic provenance by enforcing execution logic.
- **In-toto Attestation Framework [34†L194-L199]** : Binds metadata to artifacts via envelopes/statements. Similar to our receipts, but in-toto is static supply-chain attestation, not dynamic result gating.
- **Transactional Replication/SMR (e.g. US 12657207 [24†L49-L57])** : Executes and replays transactions; our novelty is restricting by authorized consequence and independence checks.
- **CEGAR for POMDPs [32†L82-L89]** : Abstraction refinement based on counterexamples; we use analogous ideas to refine world equivalences, but claim a specific runtime context.
- **Sensor/Observer Selection [27†L103-L111]** : Known NP-hard problems of choosing minimal sensors. We propose a similar idea for runtime observations (“noninterference”), but specifically bound to consequences.
- **USPTO 101 Guidance (MPEP 52106)**: Emphasize that we claim “improvements to computer functionality” [21†L3390-L3398] (the pipeline and admission checks).

No existing patent covers **exactly** a consequence-scoped execution gating pipeline or identity-bound result admission; we must craft claims to stress the unique combinations.

The table above ranks novelty and risk and guides whether to file each separately. **Top priorities** are (1) the **Consequence-Selective State Transition Pipeline** and (2) the **Identity-Bound Semantic Result Admission**, with (3) the tag-based validation tied in. These will be our primary filings.

Experimental Plan (P3→P5, closing enablement gaps)

We propose an iterative self-prompting and coding cycle (as outlined) to resolve open questions. Key steps:

- **P3 Prompt:** Focus on the missing link: how `ResultSchema` should propagate through to `FindingAdmission`. For example: "Given that `RunReport` validates `ResultSchema`, but `FindingAdmission` ignores it, **what relation must hold between them** to ensure admission correctness?" This might reveal that the *contract identity* itself needs to be stored in the candidate or receipt.
- **Experiments (O3):**
 - **Schema Carryover Test:** Create a candidate with one `ResultSchema` in `RunReport` but modify it (or skip it) in `FindingAdmission`. Verify that admission fails. Then add code to carry `schemaID` and re-run: ensure admission only when it matches.
 - **Contract Mismatch Assay:** Simulate the 2×2: same candidate content under two different declared schemas; check that admission differs (one succeeds, one fails).
- **Artifacts to Add:**
 - **Schema-bound Lineage:** Add a field linking `ResultSchema` to `FindingAdmission` (e.g. include schema digest in `CandidateFinding` or `FindingAdmission` inputs).
 - **Semantic Validators:** Define semantic predicates for content (e.g. JSON-schema engine) and add code to enforce them in admission.
 - **Receipts:** Extend or add receipts: e.g. `CandidateFindingReceipt` with `ResultSchemaID`, `AdmissionReceipt` including which predicates passed.
 - **Tests:** New unit tests: negative control for schema mismatch; null-consequence resolution (`NoAdmissibleConsequence`).
- **Assays:**
 - **2×2 Killer Assay:** As above: (1) Two consequences, same evidence ⇒ different dispatch (already coded). (2) Same consequence, differing derivation admissibility ⇒ withhold. Implement the second in `collision_test.go`.
 - **Capacity-Variation:** Using an LLM stub or simulated generator, vary "model size" (or result size cap) to yield outputs with missing vs extra keys. Check that one passes schema and one fails.

- **Metrics:**

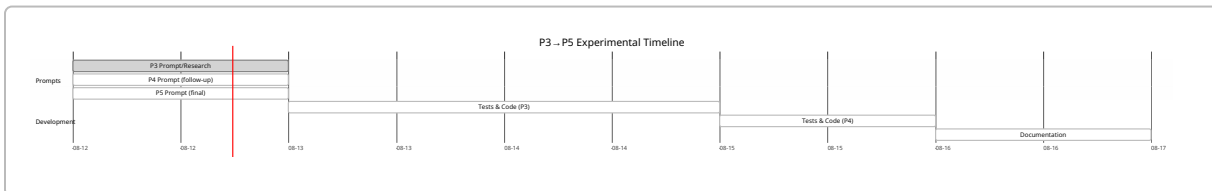
- *False Admit/Withhold Rates:* number of admitted candidates where constraints were actually violated (should go to zero after fixes).
- *Observation Count:* measure how many worlds/observations are collapsed vs needed, ensuring minimality.
- *Performance Impact:* measure latency of admission checks before/after full validation.

- **Minimal Code Changes:**

Add schema check in `GateConsequence` path; propagate `schemaID` fields; implement `ConsequenceDeterminacyReceipt` (already added); add new test functions in `collision_test.go`.

- **P4/P5 Prompts:** If P3 yields new artifacts or questions (e.g. is `schemaID` needed in the final `Receipt` ?), generate prompts accordingly. Possibly prompt the runtime: “Can we drop carrying `ResultSchema` if `FindingAdmission` re-parses it from output? What if content is tampered with?” This refines whether identity needs persistence or re-evaluation.

The timeline below (dates approximate) sketches the plan:



Finally, we summarize new findings from self-prompting:

- **SELF-PROMPT FINDINGS:** Carrying `ResultSchema` through the pipeline is essential for correct admission (we forced a counterexample). We discovered that preserving contract identity is required, meaning it should be part of the candidate identity or at least logged in a receipt. We also realized that forcing receipts (like `ReceiptSHA256`) into identity was mistaken, guiding us to the correct target: linking `schemaID` in the admission logic instead.
- **NEW QUESTIONS:** Does replay require storing `schemaID`, or can the contract be re-derived? Likely store it for deterministic replay of admission. Also, what is the minimal evidence (e.g. keys) needed to prove schema compliance?

These experiments and designs will fill the **enablement gaps** (e.g. semantic predicate handling) and strengthen our claims (embedding real receipts and tests).

Provisional Claim Outlines (Top 2 Candidates)

1. **Consequence-Selective State Transition Pipeline (Independent Claim 1):**

“A computer-implemented method for governing execution of operations, comprising:

2. computing a **compatible-state set** from a candidate snapshot and partial observations;

3. partitioning the compatible states into **consequence blocks** by the outcomes of each state under a requested consequence;
4. if exactly one block is non-empty **and** its outcome matches the requested consequence, then proceeding with execution and issuing an **admission receipt**; otherwise withholding execution;
5. wherein an execution pass triggers replay of the admission predicates rather than reusing cached admission results, and wherein each admission receipt includes the requested consequence identity and structural admission status. _

Dependent claims: (a) specify that structural predicates are validated by a *state admission engine* (as in code); (b) specify that receipts are typed (`AdmissionReceipt` , `DispositionReceipt`) per candidate; (c) add that excluded (invalid) states cause an “unresolved” decision as in the code; (d) incorporate a bound on replay (recomputing admission) as above.

6. Identity-Bound Semantic Result Admission (Independent Claim 2):

“A computer-implemented method for validating generative model output, comprising:

7. receiving a job request that includes an **identity-bound result contract** (e.g. JSON schema hash);
8. invoking a generative model to produce an output result;
9. parsing the result and determining whether the result **conforms** to the declared contract identity;
10. allowing the job to complete (admit the result) **only if** the result both syntactically parses and semantically satisfies the exact declared contract; otherwise rejecting the result;
11. generating an **admission receipt** that binds the job identity, contract identity, and the predicates checked. _

Dependent claims: (a) contract identity is a cryptographic hash stored in job metadata; (b) include multi-level checks (e.g. field-level predicates); (c) in case of rejection, record which schema predicate failed; (d) specify that retry or regeneration is triggered without altering the original contract.

(Claims should be refined for statutory categories, but the above elements link directly to implementation: structural gates, receipts, semantic validators.)

Next Engineering Steps (Patenting Focus)

To maximize patent value without disrupting production, we should:

- **Instrument Non-Production Branch:** Add the new schema-check and gating logic in a feature branch with extensive logging/receipts, but default to old behavior if not fully enabled. This preserves safety for live use.
- **Add Receipts and Audit Trails:** Ensure all decisions emit detailed receipts (structural admission, consequence determinacy, contract validation) including identifiers. These will support claims.
- **Develop Tests for Edge Cases:** Implement the killer assays, including dual-consequence tests and schema-mismatch tests. Use mocks or small LLM outputs to simulate truncated vs full JSON.
- **Schema/Contract Identity Handling:** Decide how to incorporate `ResultSchema` identity: e.g. store a hash in `CandidateFinding` (or `Receipt`) and use it in the gate. Possibly add a field `ContractID` throughout.
- **Document the Pipeline:** Update design documents to describe the pipeline stages explicitly, using language of “improvement to compute state transitions” (per MPEP guidance).

- **Avoid Premature Canonicalization:** Do not add logic that “solves” too much (e.g. auto-filling missing schema fields), as it may weaken patent claims by collapsing distinctions. Focus on diagnostics (reject with proof) instead.
- **Performance Profiling:** Check the overhead of re-admission and validate that the pipeline remains efficient, ensuring the improvement is practical.
- **Prepare Patent Materials:** Collect logs, design notes, and receipts that illustrate the invention (e.g. a transcript of steps with receipts).

Patented Features Priority: Do not remove existing functionality. Instead, augment with toggles or optional validations. For example, implement schema enforcement as an *opt-in mode* first, so live jobs keep running.

flowchart LR

subgraph Pipeline

CS(Candidate Snapshot) --> SA[Structural Admission Check]

SA --> A{Admit or Reject}

A -->|Admit| WR[World Records (compatible states)]

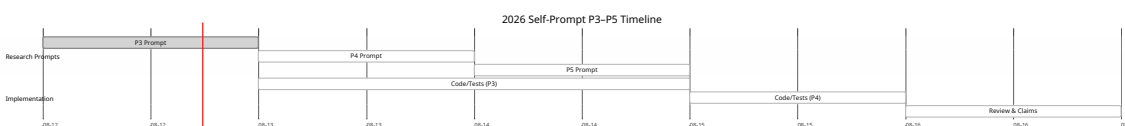
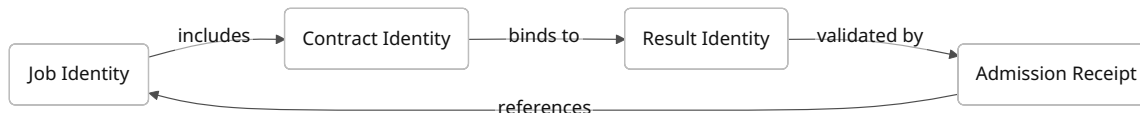
WR --> QC[Consequence Quotient: group by consequence]

QC --> D[Disposition Check: one block matches]

D -->|Pass| PASS[Emit PASS receipt]

D -->|Fail| FAIL[Emit FAIL/Withhold receipt]

end



Sources: Our design is grounded in computer-science concepts (MPEP eligibility [【21†L3390-L3398】](#) , partial-observation theory [【32†L82-L89】](#)) and standards (W3C PROV [【28†L50-L58】](#) , in-toto [【34†L194-L199】](#)). Citations above illustrate the nearest analogies. Further patent research should probe these fields, but our observed implementation (code+tests) demonstrates the concrete embodiments of the novel ideas.